

Code review — academy

Runde de review pe proiectul tău, cea mai nouă sus.

Proiect: mini-ERP de academie, C# consolă, fără framework. Regula fișierului: runda nouă se adaugă **sus**. Rundele vechi rămân dedesubt, condensate.

Runda 4 — 2026-09-15 — sha `aba4e01`

Scop: verificat commit-ul `aba4e01` „code review fixes” — ce s-a închis din cele 8  ale rundei 3 și ce s-a întâmplat cu `Enrolment`. Metodă: `dotnet build` pe codul tău, apoi o **copie** a proiectului în scratchpad, reparată strict cât să pornească, ca să pot rula scenariile. Codul tău nu a fost modificat.

Poarta de compilare — **PICĂ**

```
Build FAILED.  
1 Warning(s)  
1 Error(s)
```

A șasea oară în nouă commit-uri.

```
EnrolmentRepository.cs(6,18) CS0311: The type 'Enrolment' cannot be  
used as type parameter 'T' in the  
generic type 'Repository<T>'. There is  
no implicit reference conversion from  
'Enrolment' to 'IEntity'.  
EnrolmentRepository.cs(8,26) CS0108: 'EnrolmentRepository.FindById'  
hides inherited member  
'Repository<Enrolment>.FindById'.
```

O singură eroare. Și aici e capcana runde: **în spatele ei stăteau încă șase**.

Am făcut pe copie ce trebuia ca tipul să se lege — `Enrolment : IEntity`, un `EnrolmentTextMapper` adevărat și un constructor `: base(mapper, path)`, exact ca la `BookRepository`. Atunci au ieșit la iveală:

```

EnrolmentService.cs(26,24) CS7036: no argument given for parameter
                             'Id' of EnrolmentCreateResponse(...)
EnrolmentService.cs(57,16) CS0103: The name 'FindById' does not
                             exist in the current context
EnrolmentService.cs(81,35) CS0103: The name 'FindById' does not
                             exist in the current context
ViewStudent.cs(153,91)      CS1061: 'EnrolmentService' does not
                             contain 'GetCourseIdListByStudentId'
ViewStudent.cs(259,33)     CS1061: 'EnrolmentService' does not
                             contain 'GetEnrolmentIdByStudentAndCourseId'
ViewStudent.cs(282,40)     CS1061: 'EnrolmentService' does not
                             contain 'GetEnrolmentIdByStudentAndCourseId'

```

După ce le-am reparat și pe acestea, buildul e verde și tot ce urmează e verificat **prin rulare**.

Ce s-a închis — și e cea mai bună rundă de până acum pe restanțe

Trei runde la rând ți-am scris că restanțele se adună mai repede decât se sting. Runda asta s-a inversat:

- **B5** **ÎNCHIS** — `Admin(..., int age, int salary, ...)`, ordinea corectă în ambii constructori. Verificat prin rulare: `Age=48`, `Salary=10000`, exact cum au intrat.
- **B8** **ÎNCHIS** — toate patru locurile au acum `throw`, nu `Console.WriteLine`. `Teacher.cs:39` · `:52` · `Admin.cs:35` · `:48`.
- **B4** **ÎNCHIS** — round-trip-ul pe cele trei modele e corect acum, toate câmpurile. Rulat:

```

Teacher.ToText    -> TEACHER,3797e6e0-...,Radu,Marin,radu@gmail.com,
                    40,5000,30,parolabuna
Teacher.FromText  -> Salary=5000 WorkHours=30 Password=parolabuna

Admin.ToText      -> ADMIN,dcfb8d47-...,Alex,Grozavu,
                    gicu@hotmail.com,48,10000,gicuEgrozav
Admin.FromText    -> Age=48 Salary=10000 Password=gicuEgrozav

```

Ai numărat câmpurile și ai pus prefixul în amândouă. Asta era proba cerută, și trece.

- **M1** `UserRepository.FindById` ștersă · **M2** `email = text` (`Trim` intră) · **M3** `UserMapper` e `static` · **M4** `ToStudentUpdateResponse(Student)` · **M7** namespace-ul lui `StudentCreateRequest` reparat · **C7** cele trei `using`-uri nefolosite, scoase.

Din 8 : **3 închise pe fond**. Din 12 : **5 închise**. E prima rundă în care ai atacat lista, nu doar o linie din ea.

Ce a mers prost

Trei lucruri, și toate trei sunt de **proces**, nu de C#:

1. **Ai șters aplicația ca să treacă buildul.** `Program.Main` e gol, `ViewLogIn.Logger` e comentat integral. Buildul (dacă ar trece) ar fi verde pentru că programul nu mai există.
2. **Ai lipit fix-urile fără să ștergi liniile pe care le înlocuiau.** La `ViewStudent`, `Int32.Parse` și `Int32.TryParse` stau acum una sub alta, în toate patru locurile. Bug-ul e neschimbat, iar meniul e stricat **și** pentru input corect.
3. **Ai mutat metode fără să actualizezi apelanții** — a treia rundă la rând, acum în `EnrolmentService` și `ViewStudent`.

● Critice

B1 — Nu compilează. O eroare vizibilă, șase ascunse în spatele ei

```
Enrolments/Repositories/EnrolmentRepository.cs:6 · :8 · Enrolments/Models/Enrolment.cs:3 ·  
Enrolments/Services/EnrolmentService.cs:26 · :57 · :81 · ViewStudent.cs:153 · :259 · :282
```

CUM E ACUM: `dotnet build` raportează 1 eroare. După ce am reparat-o pe copie, au apărut alte 6, listate mai sus. Nouă linii de cod rupt, dintr-un raport care spunea „o eroare”.

DE CE: compilatorul lucrează în etape. Întâi leagă tipurile: cine moștenește pe cine, ce interfețe implementează fiecare, ce membri are fiecare clasă. Abia după ce harta asta e completă intră în corpurile metodelor, ca să verifice apelurile.

`EnrolmentRepository : Repository<Enrolment>` pică în **prima** etapă: `Repository<T>` cere `where T : class, IEntity`, iar `Enrolment ( Enrolment.cs:3 )` nu implementează `IEntity` — n-are decât un `Guid Id`, care seamănă, dar nu declară nicăieri că respectă contractul. Constrângerea nu se uită după formă, se uită după declarație.

Iar dacă `EnrolmentRepository` nu se poate lega de clasa de bază, atunci pentru compilator `EnrolmentRepository` **n-are** `Items`, **n-are** `FindById`, **n-are** `Add`. Toate apelurile către ele devin nesigure — și el nu le mai raportează, ca să nu te inunde cu erori care sunt toate ecoul primeia. De-aia numărul „1 eroare” e înșelător: **o eroare de legare a unui tip suprimă toate erorile din codul care îl folosește.**

Regula practică: când buildul pică pe o eroare de moștenire sau de constrângere generică, nu presupune că restul e sănătos. Repar-o și dă build **din nou**, până numărul rămâne 0 de două ori la rând.

Cele șase erori ascunse sunt, la rândul lor, un singur tipar: ai mutat `FindById`, `GetCourseIdListByStudentId` și `GetEnrolmentIdByStudentAndCourseId` din `EnrolmentService` în

`EnrolmentRepository` , și n-ai actualizat locurile care le chemau. Aceeași propoziție a fost scrisă la B1 în runda 2 (`ViewLogIn`) și la B1 în runda 3 (`ViewLogIn:10`). E a treia oară.

Și `EnrolmentCreateResponse` a devenit `record` pozițional cu `Id` în față, dar `EnrolmentService.cs:26` îl construiește încă prin object initializer (`new EnrolmentCreateResponse { StudentId = ... }`), care nu mai are constructor fără parametri de pe care să pornească.

FIX:

```
// Enrolment.cs
public class Enrolment : IEntity

// EnrolmentRepository.cs – constructorul, ca la BookRepository.cs:9
public EnrolmentRepository()
    : base(new EnrolmentTextMapper(),
          Path.Combine("..", "..", "..", "Data", "enrolments.txt")) { }

// EnrolmentRepository.cs – sterge FindById, e mostenita

// EnrolmentService.cs:26
return new EnrolmentCreateResponse(enrolment.Id, enrolment.StudentId,
                                   enrolment.CourseId,
                                   enrolment.CreatedAt);

// EnrolmentService.cs:57, :81
if (repository.FindById(...) != null)
```

Apoi caută **toți** apelații a ceea ce ai mutat: `grep -rn`

`"GetCourseIdListByStudentId\|GetEnrolmentIdByStudentAndCourseId" .` — sau Ctrl+F în Visual Studio, „Find All References” pe fiecare metodă mutată, înainte de commit.

B2 — Aplicația nu mai există. `Main` e gol, `Logger` e comentat integral

`Program.cs:5–7` · `ViewLogIn.cs:8` · `:21` · `:69`

CUM E ACUM: rulat pe copia care compilează:

```
$ ./stefan-academy-vanilla-charp
$ echo $?
0
```

Nu se afișează nimic. Programul pornește, nu face nimic, iese cu succes.

```
// Program.cs
public static void Main()
{
}
```

```
// ViewLogIn.cs:8
/*public void Logger()
{
    ...
    //User user = service.repository.GetByFirstAndLastName(...);
    if (user == null) { ... }
    ...
}*/
```

DE CE: merită înțeles exact ce ai făcut, pentru că e o mișcare pe care o s-o mai ai de făcut și trebuie făcută altfel.

Un bloc `/* */` nu „pune deoparte” codul — îl **șterge** din punctul de vedere al compilatorului. Textul rămâne în fișier ca să-l citești tu, dar pentru `dotnet build` acele 62 de linii nu există: nu se verifică tipurile din ele, nu se raportează erorile din ele, nu ajung în executabil. De-aia buildul tău nu mai reclamă nimic despre `ViewLogIn` — n-are ce. Semnalul „build verde”, pe care ți-l cer de cinci runde, devine **fals** în clipa în care el e verde pentru că programul lipsește.

Și e o dovadă că nu e temporar: în interiorul comentariului ai comentat **încă o linie**, `//User user = ...`. Dacă ai scoate mâine blocul mare, codul n-ar compila, pentru că `if (user == null)` de dedesubt n-ar avea niciun `user`. Ce ai comentat nu e „aproape gata” — e rupt în două locuri.

Cauza reală e B1 din runda 3, care n-a fost rezolvată, ci ocolită: `service.repository` era inaccesibil (`private`), iar metoda care trebuia să-l înlocuiască — `UserService.GetByFirstAndLastName` — n-a fost scrisă niciodată. Fix-ul era scris integral în runda 3, patru linii.

Regula, și e una de igienă: **git ține tot**. Nu ai de ce să comentezi cod ca să-l păstrezi — dacă îl ștergi și îți trebuie peste o lună, `git log -p` ți-l dă înapoi. Codul comentat nu se compilează, nu se testează, nu se

refactorizează odată cu restul, și se strică tăcut. Singurul lucru pe care îl face sigur e să ascundă că aplicația nu merge.

FIX:

```
// UserService.cs – metoda care lipseste de doua runde
public User GetByFirstAndLastName(string firstName, string lastName)
{
    return repository.GetByFirstAndLastName(firstName, lastName);
}

// ViewLogIn.cs – scoate /* */, si apoi:
User user = service.GetByFirstAndLastName(firstName, lastName);

// Program.cs – pune Main-ul inapoi
public static void Main()
{
    try
    {
        ViewLogIn logIn = new();
        logIn.Logger();
    }
    catch (ArgumentException text)
    {
        Console.WriteLine(text);
    }
}
```

B3 — Ai lipit fix-ul peste linia stricată, în loc s-o înlocuiești. Acum meniul e stricat și pentru input corect

ViewStudent.cs:35 vs :37 · :70 vs :72 · :101 vs :103 · :127 vs :129

CUM E ACUM: codul, în toate patru locurile:

```
tasta = Int32.Parse(Console.ReadLine());

if (!Int32.TryParse(Console.ReadLine(), out taste))
{
    InputGresit();
    continue;
}
```

Rulat pe copie, cu `ViewStudent` pornit direct pe un student:

```
$ printf '0\n' | ./stefan-academy-vanilla-charp
Apasati taste 0 pentru a iesi
Apasati taste 1 pentru a lucra cu CARTI
Apasati taste 2 pentru a lucra cu CURSURI
Apasati taste 3 pentru a vedea statisticile
Ati introdus un caracter nepermis!
```

```
$ printf 'a\n' | ./stefan-academy-vanilla-charp
...
System.FormatException: The input string 'a' was not in a correct format.
   at ViewStudent.Viewer() in ViewStudent.cs:line 35
```

Am apăsât `0`, o tastă perfect validă, și mi-a spus „caracter nepermis”. Am apăsât `a` și a crăpat exact ca în rundele 1, 2 și 3, la aceeași linie.

DE CE: `Console.ReadLine()` nu „se uită” la ce a tastat utilizatorul — el **consumă** o linie din fluxul de intrare și o scoate de acolo. Fiecare apel cere o linie nouă. Tu ai două apeluri, deci programul cere **două** linii pentru o singură alegere de meniu.

De aici, cele două comportamente pe care le-ai văzut:

- Am dat `a` : prima citire îl ia, `Int32.Parse` crapă imediat. Linia 37 nu se execută niciodată — `TryParse` -ul scris exact pentru cazul ăsta e cod mort, la două rânduri sub locul unde programul moare.
- Am dat `0` : prima citire îl ia și `taste` devine 0 corect. A doua citire găsește fluxul gol, `TryParse` pe `null` întoarce `false`, și pleacă pe `InputGresit()`.

Bug-ul nu e că `TryParse` e greșit — e scris perfect. Bug-ul e că un fix care **înlocuiește** ceva a fost aplicat ca și cum ar **adăuga** ceva. Când un review îți dă un bloc de cod, întrebarea dinainte de a-l lipi e mereu: ce

linie moare în locul lui? Dacă nu moare niciuna, fie ai înțeles greșit fix-ul, fie nu era un fix.

Al doilea lucru pe care îl arată: **nu l-ai rulat**. Un singur `0` apăsat o dată ar fi arătat că meniul nu mai merge. Fix-ul are patru copii în fișier, deci a fost lipit de patru ori fără ca aplicația să fie pornită între timp — ceea ce se leagă direct de B2, pentru că aplicația nici nu mai putea porni.

FIX: șterge linia `Int32.Parse`, păstrează blocul `TryParse`. În toate patru locurile:

```
if (!Int32.TryParse(Console.ReadLine(), out tasta))
{
    InputGresit();
    continue;
}
```

B4 — (runda 2, a patra oară) Aplicația tot nu poate citi `users.txt` — iar acum datele și codul se contrazic pe față

`Users/Mappers/UserTextMapper.cs:10` · `:17` · `Data/users.txt` · `Users/Models/Student.cs:5` · `Teacher.cs:6` · `Admin.cs:5`

CUM E ACUM: am restaurat `Main` și `Logger` pe copie și am pornit:

```
System.FormatException: Unrecognized Guid format.
   at UserTextMapper.FromText(String text) in UserTextMapper.cs:17
   at Repository`1.Read() in Repository.cs:63
   at Repository`1..ctor(ITextMapper`1, ...) in Repository.cs:13
   at UserRepository..ctor() in UserRepository.cs:10
   at ViewLogIn.Logger() in ViewLogIn.cs:10
```

Identice cu rundele 2 și 3. Apoi am scos prefixele din `users.txt`, ca să treacă de `Guid.Parse`, și m-am logat cu un student:

```
=====LOG IN=====
Numele: Prenumele: System.ArgumentException: Error on user type
   at ViewLogIn.Logger() in ViewLogIn.cs:line 67
```

DE CE: runda asta ai mutat **datele** spre soluție, dar nu și **codul**. `Data/users.txt` are acum prefixe:

```
STUDENT,d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
ADMIN,9c99cd02-...,Alex,Grozavu,gicu@hotmail.com,48,10000,gicuEgrozav
```

iar `UserTextMapper.FromText ( :17 )` face în continuare `Guid.Parse(cuv[0])` — adică încearcă să citească `"STUDENT"` ca pe un GUID. Înainte cele două erau doar nesincronizate; acum sunt **garantat incompatibile**: fișierul în format nou, cititorul în format vechi. Prima linie a fișierului oprește aplicația.

Iar cele șase metode pe care le-ai scris și le-ai reparat corect runda asta (vezi B4 închis, round-trip verde) tot nu pot fi chemate de nimeni, din motivul explicat la B3 runda 3: `Student` implementează `ITextMapper<Student>`, iar `Repository<User>` cere un `ITextMapper<User>`. Invarianța generică — `Cutie<Pisică>` nu e `Cutie<Animal>` — nu se schimbă pentru că metodele dinăuntru sunt corecte. **Conținutul e bun, cablajul lipsește.**

Și de-aia, chiar după ce scoți prefixele cu mâna, `FromText` întoarce un `User` gol de tip, cele trei `as` din `ViewLogIn` dau toate `null`, și pică pe `"Error on user type"`. E T3, Factory Method, și e singurul lucru care stă între tine și o aplicație care pornește.

FIX: exact ce scria la B3 runda 3 — ramificarea pe `cuv[0]` în `UserTextMapper`, iar cele șase metode (deja scrise corect!) mutate acolo ca `private static`:

```
public User FromText(string text)
{
    string[] cuv = text.Split(',');

    if (cuv[0] == "STUDENT") { return StudentFromText(cuv); }
    if (cuv[0] == "TEACHER") { return TeacherFromText(cuv); }
    if (cuv[0] == "ADMIN")   { return AdminFromText(cuv);   }

    throw new ArgumentException("Tip de user necunoscut: " + cuv[0]);
}
```

Partea grea o ai deja făcută. Ce lipsește e mutarea și `if`-ul.

B5 — `Save()` tot rade salariile și parolele — iar acum șterge și prefixele pe care tocmai le-ai pus în fișier

Users/Mappers/UserTextMapper.cs:10 · Common/Repository.cs:48

CUM E ACUM: am creat pe copie un profesor și un student prin serviciile tale, apoi am chemat `Save()` pe `UserRepository`. Conținutul lui `users.txt` după:

```
d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
...
9c99cd02-...,Alex,Grozavu,gicu@hotmail.com,48
7ab8229d-...,Radu,Marin,radu@gmail.com,40
```

Adminul și-a pierdut `10000` și `gicuEgrozav`. Profesorul e pe disc fără salariu, fără ore, fără parolă. Și **toate liniile au rămas fără prefix** — deci un singur `Save()` anulează exact modificarea pe care ai făcut-o tu în `Data/users.txt` runda asta.

DE CE: `Repository<T>.Save ( Repository.cs:48 )` scrie `mapper.ToText(items[i])`, iar `mapper` e cel primit în constructor — `UserTextMapper`. Al lui `ToText ( :10 )` produce cinci câmpuri, fără prefix:

```
return item.Id + "," + item.FirstName + "," + item.LastName
    + "," + item.Email + "," + item.Age;
```

Parametrul e `User item`, deci accesează doar ce știe un `User`. Salariul unui `Teacher` există în obiectul din memorie, dar `ToText` nu are cum să-l vadă: **tipul static al parametrului decide ce câmpuri sunt vizibile**, nu tipul real al obiectului. Un `Teacher` trecut printr-o variabilă `User` e în continuare un `Teacher` complet — dar codul care se uită prin fereastra `User` vede doar cinci câmpuri.

Asta face din `Save()` o operație **cu pierdere**: fiecare salvare e o trecere prin filtrul `User`, iar ce nu încapă se pierde definitiv. Nu e un bug care strică afișarea; e un bug care distruge date pe disc. De-aia, în runda 3, `Save()` a fost pus explicit **după** T3 pe lista de ordine: cât timp scriitorul nu știe de tipuri, orice salvare taie.

FIX: aceeași ramificare ca la B4, dar în sens invers, în `ToText` :

```
public string ToText(User item)
{
    if (item is Student s) { return StudentToText(s); }
    if (item is Teacher t) { return TeacherToText(t); }
    if (item is Admin a)   { return AdminToText(a);   }

    throw new ArgumentException("Tip de user necunoscut");
}
```

DoD, și e același de trei runde: pui un TEACHER în mijlocul lui `users.txt`, pornești, te loghezi cu el, salvezi, repornești — profesorul e tot profesor și are tot parola.

B6 — Garda pe email a intrat doar la studenți. La profesori și admini, trei apeluri identice trec toate trei

`Users/Services/UserService.cs:42` · `:55` (vs `:28`, care e corect)

CUM E ACUM: am chemat de trei ori `CreateTeacher` cu exact aceleași date, apoi de trei ori `CreateStudent` :

```
=== CreateTeacher de 3 ori cu aceleasi date ===
apel 1: acceptat, Id=7ab8229d-...
apel 2: acceptat, Id=c71140d8-...
apel 3: acceptat, Id=791ef7ac-...

=== CreateStudent de 3 ori cu acelasi email ===
apel 1: acceptat, Id=60dd5dc4-...
apel 2: respins -> Studentul se afla deja in baza de date
apel 3: respins -> Studentul se afla deja in baza de date
```

`CreateStudent`  face exact ce trebuie. `CreateTeacher` (`:42`) și `CreateAdmin` (`:55`) au rămas cu `repository.FindById(teacher.Id)`, care e mereu `null` pentru că `Id`-ul tocmai s-a născut pe linia de deasupra.

DE CE: mecanismul l-am explicat runda trecută și l-ai aplicat corect — la un singur loc din trei. Ce merită explicat acum e **cum se citește un review**, pentru că asta e al treilea review la rând în care se întâmplă același lucru.

Fiecare constatare are două părți: un **unde** (`file:line`, ca să poți deschide fișierul) și o **regulă** (ce anume e greșit ca raționament). Adresa e doar punctul din care ți-am arătat regula — nu e lista completă a locurilor în care regula e încălcată. Runda trecută, B6 se termina cu propoziția: „*Același tratament la `CreateTeacher` și `CreateAdmin` — sunt trei locuri cu aceeași greșeală.*” Adresele erau scrise. Ai reparat-o pe prima.

Obiceiul care încheie asta definitiv, și îți ia 30 de secunde: după fiecare fix, **caută în tot proiectul tiparul pe care tocmai l-ai reparat**. Aici: `Ctrl+Shift+F` pe `FindById` — patru rezultate, trei dintre ele în `UserService`, două greșite. Nu trebuie să ghicești; trebuie să cauți.

Semn secundar, în același loc: `UserService.cs:26` e o linie goală rămasă acolo unde era codul mutat. Când un fix lasă în urmă o gaură, înseamnă că a fost aplicat prin lipire, nu prin editare — și lipirea e exact ce a produs și B3.

FIX:

```
// CreateTeacher si CreateAdmin, la fel ca CreateStudent
if (repository.GetByEmail(request.Email) != null)
{
    throw new ArgumentException("Exista deja un user cu acest email");
}

Teacher teacher = UserMapper.ToTeacher(request);
repository.Add(teacher);
```

B7 — `EnrolmentService` citește fișierul de două ori și ține două liste paralele; `repository` nu e folosit niciodată

`Enrolments/Services/EnrolmentService.cs:9` · `:10` · `:14` · `:61` · `:126` ·
`Enrolments/Mappers/EnrolmentTextMapper.cs:3` · `EnrolmentMapper.cs:3`

CUM E ACUM: În aceeași clasă, două surse de adevăr:

```
EnrolmentRepository repository = new();           // :9 -> citește
                                                    //      enrolments.txt
private readonly List<Enrolment> enrolments = new(); // :10

public EnrolmentService()
{
    ReadEnrolments();                               // :14 -> citește
}                                                    //      enrolments.txt
                                                    //      inca o data
```

`grep -n "repository" EnrolmentService.cs` întoarce **o singură linie**: declarația de la `:9`. Câmpul e construit, citește tot fișierul de pe disc, și apoi nu e folosit nicăieri. `CreateEnrolment` (`:61`) adaugă în `enrolments`. `Save()` (`:126`) scrie `enrolments`. `UpdateEnrolment`, `DeleteEnrolment` — tot `enrolments`.

Iar `EnrolmentTextMapper` și `EnrolmentMapper` sunt clase **goale**, cu corpul `{ }`, comise ca atare.

DE CE: ăsta e un refactor oprit la jumătate, și merită văzut ce anume îl face periculos — pentru că, spre deosebire de o eroare de compilare, tiparul ăsta compilează și rulează perfect.

Ai două copii în memorie ale aceluiași fișier. Fiecare scriere merge într-una singură. De aici înainte, răspunsul la orice întrebare depinde de **din care listă citești**: `repository.FindById` ar vedea starea de la pornire, `enrolments` ar vedea starea curentă. Cele două se depărtează una de alta cu fiecare operație, tăcut, și n-ai niciun semnal că se întâmplă. Bug-urile de tipul ăsta se manifestă mult mai târziu, ca „am șters înscrierea și tot apare” — și se caută ore.

Un `Repository<T>` există exact ca să existe **un singur** loc care știe unde stau datele. În clipa în care serviciul ține și el o listă și își citește și el fișierul, repository-ul nu mai e un repository, e un al doilea cache.

Ai deja modelul corect în propriul tău proiect, la 40 de linii distanță: `BookRepository.cs:9` are constructorul cu `base(mapper, path)` și zero metode duplicate, iar `BookService` primește repository-ul din afară și nu ține nicio listă proprie. `EnrolmentRepository` trebuia să fie copia lui.

FIX:

1. `EnrolmentTextMapper` — scrie-l, e simetricul lui `BookTextMapper` :

```
public class EnrolmentTextMapper : ITextMapper<Enrolment>
{
    public string ToText(Enrolment item)
    {
        return item.Id + "," + item.StudentId + "," + item.CourseId
            + "," + item.CreatedAt.ToString("yyyy-MM-dd");
    }

    public Enrolment FromText(string text)
    {
        return new Enrolment(text);
    }
}
```

2. `EnrolmentService` : șterge `enrolments` , `ReadEnrolments()` , `EnrolmentsListToString()` și `Save()` -ul propriu. Primește repository-ul în constructor, ca `BookService` , și cheamă `repository.Save()` .
 3. `EnrolmentMapper` — sau îl scrii (mută acolo cele trei metode de mapare din `EnrolmentService:19-37`), sau îl ștergi. O clasă goală comisă e o promisiune nescrisă.
-

🟡 Importante

- **M1 — `EnrolmentRepository.FindById` ascunde metoda moștenită.** `EnrolmentRepository.cs:8` · `warning CS0108` . E exact greșeala pe care ai **șters-o** din `UserRepository` runda asta (M1 ✅), refăcută identic în fișierul nou. `Repository<T>.FindById` (`Repository.cs:21`) face același lucru.
- **M2 — `EnrolmentRepository` n-are constructor.** `:6` — deci nu poate trimite mapper și path la clasa de bază. Modelul e `BookRepository.cs:9` .
- **M3 — `EnrolmentMapper` și `EnrolmentTextMapper` sunt clase goale comise.** Vezi B7.
- **M4 — (R3 M2, jumătate) verificarea de lungime la `Email` e tot înaintea lui `Trim`.** `User.cs:98` verifică 7–40 pe `value` , iar `Trim` -ul vine abia la `:103` . ✅ `email = text` e reparat. Mută `Trim` -ul sus, înaintea tuturor verificărilor.
- **M5 — (R3 M6, NEATINS) `DeleteUser` nu verifică dacă userul există.** `UserService.cs:108–112` . Reprodus: `DeleteUser(Guid.NewGuid())` → „a raportat succes, fara exceptie”. Frații ei, `Update*` , verifică și aruncă.
- **M6 — (R3 M5, NEATINS) nimic nu cheamă `Save()` pentru useri.** La închiderea aplicației se pierde tot. (E T2, Observer — după T3.)
- **M7 — (R1, NEATINS) `InscriereCurs` ocolește serviciul.** `ViewStudent.cs:265` — `enrolmentService.Enrolments.Add(...)` direct în lista internă. Legat de B7: exact lista care nu e a repository-ului.
- **M8 — (R1, NEATINS) modificarea unei cărți nu confirmă nimic.** `ViewStudent.cs:222` — `response` atribuit și nefolosit.
- **M9 — (R1, NEATINS) login-ul de admin/profesor iese tăcut.** `ViewLogIn.cs:39` / `:53` — `//viewAdmin` , `//viewTeacher` , acum în interiorul blocului comentat.
- **M10 — (R1, NEATINS) serviciile se instanțiază singure.** `ViewStudent.cs:15` (`CourseService`) și `:17` (`EnrolmentService`) își citesc fișierul din constructor. `BookService` (`:16`) ✅ primește repository-ul — el e modelul.
- **M11 — (R1, NEATINS) `GetEnrolmentIdByStudentId` e cod mort.** `EnrolmentRepository.cs:17` — mutată în repository, tot nechemată de nimeni.
- **M12 — `EnrolmentUpdateResponse(Guid id, ...)` — `id` cu literă mică, singurul din cele 16 DTO-uri.** `EnrolmentUpdateResponse.cs:3` .

🟢 Cleanups

- **C1 — (R1, a patra oară) tot nu există `.gitignore`.** `bin/` și `obj/` apar ca *untracked* la mine, acum, după build.
- **C2 — 62 de linii nou comentate** în `ViewLogIn.cs:8–69` , peste cele 28 din `User.cs:136–163` . `git` ține tot; șterge-le.

- **C3 — (R1, NEATINS)** `Data/students.txt` e mort. `grep -rn students.txt` peste tot codul: 0 rezultate.
 - **C4 — (R1, a patra oară)** `Console.WriteLine(aux);` rămas din depanare, `ViewStudent.cs:302`.
 - **C5 — (R1, NEATINS)** parole în clar în `Data/users.txt` (`gicuEgrozav`).
 - **C6 — 35 de warnings** pe copia reparată (pe codul tău nu se pot număra, buildul se oprește la eroare). Majoritatea `CS8600` / `CS8604` / `CS8603` — `null` care circulă prin cod fără să fie declarat ca atare.
 - **C7 — linie goală rămasă din mutare,** `UserService.cs:26`.
 - **C8 — fișiere fără linie goală la final** (`\ No newline at end of file`), tot.
-

Starea celor din runda 3

#	Runda 3	Acum	Cum am verificat
B1	nu compilează (4 erori)	🔴 redeschis — 1 vizibilă + 6 ascunse, în <code>Enrolment</code>	<code>dotnet build</code> , de două ori
B2	aplicația nu pornește (<code>Guid.Parse</code>)	🔴 NEATINS (R4 B4) — plus datele au trecut în format nou	rulat: același <code>FormatException</code>
B3	metodele din modele nu pot fi chemate; <code>Save()</code> rade parole	🔴 NEATINS (R4 B4, B5)	rulat: <code>Save()</code> → 5 câmpuri
B4	<code>FromText</code> contrazice <code>ToText</code>	✅ ÎNCHIS pe toate trei tipurile	rulat: round-trip verde
B5	<code>Admin</code> : vârsta și salariul inversate	✅ ÎNCHIS	rulat: <code>Age=48</code> <code>Salary=10000</code>
B6	garda de duplicat nu se declanșează	⚠️ o treime — <code>CreateStudent</code> ✅, <code>CreateTeacher</code> / <code>CreateAdmin</code> 🔴	rulat: 3 apeluri identice, toate acceptate
B7	<code>Int32.Parse</code> pe tastă	🔴 în răutăți (R4 B3) — fix lipit peste linia stricată	rulat: <code>0</code> → „caracter nepermis”; <code>a</code> → crash
B8	<code>Console.WriteLine</code> în loc de <code>throw</code>	✅ ÎNCHIS , toate patru	citit + rulat
M1	<code>UserRepository.FindById</code> ascunde	✅ ÎNCHIS — 🔴 refăcut în <code>EnrolmentRepository</code>	<code>warning CS0108</code>
M2	<code>Email</code> fără <code>Trim</code>	⚠️ jumătate — <code>email = text</code> ✅, lungimea tot pre- <code>Trim</code>	rulat: lungime 13
M3	<code>UserMapper</code> nu e <code>static</code>	✅ ÎNCHIS	
M4	<code>ToStudentUpdateResponse(User)</code>	✅ ÎNCHIS	
M5	<code>Save()</code> nechemat pentru useri	🔴 NEATINS (M6)	

#	Runda 3	Acum	Cum am verificat
M6	<code>DeleteUser</code> fără verificare	🔴 NEATINS (M5)	rulat: succes pe id inexistent
M7	namespace <code>StudentCreateRequest</code>	✅ ÎNCHIS	
M8– M12	cod mort, <code>InscriereCurs</code> , confirmare, login tăcut, servicii duble	🔴 NEATINSE	<code>grep</code>
C1	cod comentat	🔴 înărăutățit — +62 de linii	
C2– C5	<code>.gitignore</code> , <code>students.txt</code> , <code>aux</code> , parole	🔴 NEATINSE	
C6	warnings	43 → 44 → 35 (pe copie)	
C7	<code>using</code> -uri nefolosite	✅ ÎNCHIS	

Din 8 🔴 : 3 închise, 1 pe o treime, 4 deschise. Din 12 🟡 : 5 închise, 1 jumătate. Din 9 🟢 : 1 închis.

E prima rundă cu bilanț pozitiv pe restanțe, și pe cele mai grele dintre ele: round-trip-ul, ordinea la `Admin`, `throw` -urile. Ce s-a stricat în schimb e **procesul**: aplicația a fost comentată ca să treacă buildul, iar buildul n-a fost rulat înainte de commit — altfel cele 7 erori s-ar fi văzut.

Before / After pentru cele

#	Acum	Cum ar trebui
B1	<code>class Enrolment</code>	<code>class Enrolment : IEntity</code>
B1	<code>class EnrolmentRepository :</code> <code>Repository<Enrolment></code> fără constructor	<code>public EnrolmentRepository() :</code> <code>base(new EnrolmentTextMapper(),</code> <code>Path.Combine(...,"enrolments.txt")) {</code> <code>}</code>
B1	<code>if (FindById(newEnrolment.Id) !=</code> <code>null)</code>	<code>if</code> <code>(repository.FindById(newEnrolment.Id)</code> <code>!= null)</code>
B1	<code>new EnrolmentCreateResponse {</code> <code>StudentId = ..., CourseId = ... }</code>	<code>new EnrolmentCreateResponse(e.Id,</code> <code>e.StudentId, e.CourseId, e.CreatedAt)</code>
B2	<code>public static void Main() { }</code>	<code>Main</code> cu <code>new ViewLogIn().Logger()</code> în <code>try / catch</code> , ca înainte
B2	<code>/*public void Logger() { ... }*/</code>	fără <code>/* */</code> , cu <code>service.GetByFirstAndLastName(...)</code>
B3	<code>tasta =</code> <code>Int32.Parse(Console.ReadLine());</code> <code>if</code> <code>(!Int32.TryParse(Console.ReadLine(),</code> <code>out tasta))</code>	doar <code>if</code> <code>(!Int32.TryParse(Console.ReadLine(),</code> <code>out tasta)) { InputGresit();</code> <code>continue; }</code>
B4	<code>UserTextMapper.FromText</code> → <code>new</code> <code>User(Guid.Parse(cuv[0]), ...)</code>	ramificare pe <code>cuv[0]</code> → <code>Student</code> / <code>Teacher</code> / <code>Admin</code> (T3)
B5	<code>ToText</code> → <code>Id + "," + FirstName + ...</code> <code>+ Age</code> (5 câmpuri)	<code>if (item is Teacher t) return</code> <code>TeacherToText(t);</code> ... cu prefix și toate câmpurile
B6	<code>Teacher teacher =</code> <code>ToTeacher(request);</code> <code>if (repository.FindById(teacher.Id)</code> <code>!= null)</code>	<code>if</code> <code>(repository.GetByEmail(request.Email)</code> <code>!= null) { throw ... }</code> <code>Teacher teacher =</code> <code>ToTeacher(request);</code>

#	Acum	Cum ar trebui
B7	<pre>EnrolmentRepository repository = new(); private readonly List<Enrolment> enrolments = new();</pre>	doar repository-ul, primit în constructor; lista internă și <code>ReadEnrolments()</code> dispar

Întrebări de verificare

- B1.** `dotnet build` ți-a spus „1 Error”. După ce am reparat acea eroare, au apărut încă șase. Explică de ce compilatorul nu ți le-a arătat de la început. Ce trebuie să faci, ca obicei, după fiecare eroare de moștenire sau de constrângere generică?
- B3.** Ai pus `TryParse` dedesubtul lui `Parse`, în loc să-l pui în locul lui. Apăs `0` — o tastă validă — și programul îmi spune „caracter nepermis”. Explică, pas cu pas, ce se întâmplă cu cele două apeluri `Console.ReadLine()` și de unde vine mesajul. Apoi: câte linii de input cere acum meniul pentru o singură alegere?
- B5.** `UserTextMapper.ToText(User item)` primește un obiect care, la rulare, chiar **este** un `Teacher` complet, cu salariu și parolă. De ce nu poate scrie salariul? Răspunde despre tipul parametrului, nu despre tipul obiectului.
- B6.** Ai reparat `CreateStudent` corect și n-ai atins `CreateTeacher` / `CreateAdmin`, deși runda trecută scria explicit „sunt trei locuri cu aceeași greșeală”. Nu te întreb de ce. Te întreb: **ce cauți în proiect, concret, imediat după ce ai terminat un fix?** Dă comanda sau combinația de taste.
- B2 / proces.** Ai comentat `Logger()` și ai golit `Main`. Presupunând că voiai să pui codul deoparte până rezolvi B1 din runda 3 — care era varianta corectă, care nu ascunde faptul că aplicația nu merge? (Indiciu: ce unealtă îți păstrează oricum tot ce ai șters vreodată?)

Ce urmează

Ordinea contează mai mult ca oricând, pentru că acum nimic nu se poate verifica prin rulare.

- Fă-l să compileze** — B1. `Enrolment : IEntity`, constructorul lui `EnrolmentRepository`, `EnrolmentTextMapper` scris, apelanții actualizați. Dă build **de două ori**: prima reparație scoate la iveală restul.

2. **Pune aplicația înapoi** — B2. `Main` + `Logger` + `UserService.GetByFirstAndLastName` . Fără asta, niciun fix nu se poate proba.
3. **Șterge cele 4 linii `Int32.Parse`** — B3. Două minute. Apoi pornește meniul și apasă `0` .
4. **T3, Factory Method** — B4 + B5. Metodele sunt deja scrise și corecte; le muți în `UserTextMapper` și adaugi `if` -ul pe `cuv[0]` , în ambele sensuri. DoD: TEACHER în mijlocul lui `users.txt` → login → `Save()` → repornire → profesorul e tot profesor, cu parolă.
5. **B6** — două locuri, copiate din `CreateStudent` .
6. **B7** — `EnrolmentService` pe repository, o singură listă.

Un commit per pas. **`dotnet build` verde înainte de fiecare commit** — e a șasea oară când o scriu, și singura regulă din review care nu costă nimic.

Și un obicei nou, pentru runda 5, în locul celui de sus: **după fiecare fix, caută tiparul în tot proiectul** (`Ctrl+Shift+F`), înainte de a trece la următorul. Din cele 4  rămase deschise, trei sunt același fix aplicat într-un singur loc din două sau trei.

Runda 3 — 2026-09-09 — sha `70360d6`

Verificat commit-ul „more user fixes”. Buildul a picat cu 4 erori. Partea de fond s-a mișcat serios (B4 din R2 închis pe bune), dar `Student` / `Teacher` / `Admin` au primit `ITextMapper<T>` — șase metode pe care nimeni nu le putea chema.

#	Constatare	Stare azi
B1	nu compilează: <code>ViewLogIn:10</code> , <code>:21</code> , <code>Teacher.cs:87</code> — apelanți neactualizați după mutare	🔴 redeschis (R4 B1), același tipar
B2	aplicația nu pornește; după reparare, nici studenții nu se pot loga	🔴 NEATINS (R4 B4)
B3	<code>ITextMapper<Student></code> nu e <code>ITextMapper<User></code> → cele 6 metode noi, necheamabile; <code>Save()</code> rade parole	🔴 NEATINS (R4 B4, B5)
B4	fiecare <code>FromText</code> contrazice <code>ToText</code> -ul de deasupra (prefix + câmpuri lipsă)	✅ ÎNCHIS — round-trip verde pe toate trei
B5	<code>Admin : salary</code> înaintea lui <code>age</code> → admin de 10000 de ani cu salariul 48	✅ ÎNCHIS
B6	<code>if (FindById(student.Id) != null)</code> — mereu falsă, <code>Id</code> proaspăt generat	⚠️ o treime — <code>CreateStudent</code> ✅ , <code>CreateTeacher</code> / <code>CreateAdmin</code> 🔴 (R4 B6)
B7	<code>Int32.Parse</code> pe tastă, 4 locuri	🔴 înrăutățit (R4 B3) — <code>TryParse</code> lipit sub <code>Parse</code>
B8	<code>Console.WriteLine</code> în loc de <code>throw</code> la <code>Salary</code> / <code>Password</code>	✅ ÎNCHIS, toate patru
M1	<code>UserRepository.FindById</code> ascunde metoda moștenită	✅ ÎNCHIS — refăcut în <code>EnrolmentRepository</code> (R4 M1)
M2	<code>Email</code> nu se curăță de spații	⚠️ jumătate — <code>email = text</code> ✅ , lungimea tot pre- <code>Trim</code> (R4 M4)
M3	<code>UserMapper</code> nu e <code>static</code>	✅ ÎNCHIS

#	Constatare	Stare azi
M4	<code>ToStudentUpdateResponse(User)</code> față de frații ei	✅ ÎNCHIS
M5	nimic nu cheamă <code>Save()</code> pentru useri	❌ NEATINS (R4 M6)
M6	<code>DeleteUser</code> nu verifică dacă userul există	❌ NEATINS (R4 M5)
M7	<code>StudentCreateRequest</code> în alt namespace	✅ ÎNCHIS
M8–M12	cod mort, <code>InscriereCurs</code> , confirmare lipsă, login tăcut, servicii duble	❌ NEATINSE (R4 M7–M11)
C1–C9	cod comentat, <code>.gitignore</code> , <code>students.txt</code> , <code>aux</code> , parole, warnings, <code>using</code> -uri, newline, naming	✅ C7; C1 ❌ înrăutățit (+62 de linii); restul NEATINSE

Fix-urile complete, cu cod, pentru fiecare dintre cele de mai sus sunt în runda 4 (pentru cele încă deschise) — iar mecanismele explicate atunci rămân valabile: invarianța generică la B3, round-trip-ul la B4, „`int` e `int`” la B5, identitate vs. date la B6.

Runda 2 — 2026-09-09 — sha `3c1877d`

Verificat pasul 3 din Faza 1 (`User` → T0.5). Buildul a picat cu 16 erori (6 vizibile, 10 ascunse în spatele lor). Pașii 1 și 2 — `Course` și `Enrolment` — sări.

#	Constatare	Stare azi
B1	nu compilează: metode duplicate în <code>UserMapper</code> + DTO-uri șterse fără apelanți	🔴 redeschis (R3 B1)
B2	<code>UserTextMapper.FromText</code> face <code>Guid.Parse</code> pe prefixul de tip → aplicația nu pornește	🔴 NEATINS (R3 B2)
B3	<code>FromText</code> întoarce mereu <code>User</code> → adminul devine student, <code>Save()</code> rade parola și salariul	🔴 NEATINS (R3 B2, B3)
B4	<code>CreateUser : if (request != null)</code> mereu adevărat → orice cerere produce un <code>Student</code>	✅ ÎNCHIS
B5	<code>Salary / Password</code> în <code>Teacher</code> și <code>Admin</code> : <code>Console.WriteLine</code> în loc de <code>throw</code>	🔴 NEATINS (R3 B8)
B6	<code>Int32.Parse</code> pe tastă în <code>ViewStudent</code>	🔴 NEATINS (R3 B7)
M1	<code>UserRepository.FindById</code> ascunde metoda moștenită (<code>CS0108</code>)	🔴 NEATINS
M2	<code>Email</code> nu se face <code>Trim</code> , deși <code>FirstName / LastName</code> da	🔴 NEATINS
M3	<code>UserMapper</code> nu e <code>static</code> ; <code>ToTeacher</code> întoarce <code>User</code>	⚠️ jumătate
M4	ordinea argumentelor: <code>Teacher</code> greșit, <code>Admin</code> — capcană anunțată	⚠️ <code>Teacher</code> ✅ / <code>Admin</code> 🔴 (R3 B5)
M5	<code>StudentCreateRequest</code> în alt namespace decât frații lui	🔴 NEATINS

#	Constatare	Stare azi
M6–M10	cod mort, <code>InscriereCurs</code> ocolește serviciul, modificarea cărții nu confirmă, login admin/profesor tăcut, servicii instanțiate de mai multe ori	🔴 NEATINS
C1–C7	cod comentat, <code>.gitignore</code> , <code>students.txt</code> mort, <code>Console.WriteLine(aux)</code> , parole în clar, warnings, <code>using</code> -uri	⚠️ C1 parțial; restul NEATINSE

Runda 1 — 2026-09-08 — sha `79abec3`

Verificat T0.0–T0.4 din `TASKS.md`. Buildul a trecut. **5/5 task-uri închise pe fond.**

#	Constatare	Stare azi
T0.0–T0.4	nu compilează · „Vezi cursurile” crapă · modificarea orfanizează cartea · <code>createdAt</code> ignorat · <code>CreateUser</code> face mereu un <code>User</code> gol	✅ toate 5 închise
B1	<code>Contains</code> la căutarea după nume → Enter pe gol înseamnă „primul din listă”	✅ ÎNCHIS
B2	<code>null</code> strecurat în lista de cursuri	✅ ÎNCHIS
B3	profesorul salvat nu mai poate fi citit (<code>ToText</code> ≠ constructorul din text)	❌ reapărut de 3 ori în modele (R3 B4)
B4	<code>Int32.Parse</code> pe tastă	❌ NEATINS de 2 runde (R3 B7)
M1	<code>Create</code> / <code>Update</code> din modele = cod mort	✅ șterse
M2	<code>BookUpdateRequest</code> cere o dată ignorată	✅ ÎNCHIS de T0.5
M3	<code>GetEnrolmentIdByStudentId</code> cod mort	❌ NEATINS
M4	modificarea cărții nu confirmă nimic	❌ NEATINS
M5	<code>User newUser = new();</code> inutil	✅ ÎNCHIS
M6	<code>InscriereCurs</code> ocolește serviciul	❌ NEATINS
M7	login admin/profesor iese tăcut	❌ NEATINS
M8	serviciile se instanțiază de mai multe ori	⚠️ <code>BookService</code> ✅, restul ❌

#	Constatare	Stare azi
C1-C7	<code>Console.WriteLine(aux)</code> · <code>.gitignore</code> · <code>students.txt</code> · parole în clar · warnings · <code>override</code> gol · <code>using</code> -uri	✅ C6; restul ❌



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro